Project Report – Distributed Cinema Reservation System

1. Description

The project implements a distributed cinema seat reservation system using a 3-node MongoDB Replica Set orchestrated with Docker Compose. The goal was to build a concurrent-safe booking service that demonstrates the principles of distributed databases: replication, automatic failover, and atomic write operations.

The system exposes two user-facing interfaces:

- Console application (app.py) – a terminal menu for viewing, booking, and updating the 70 seats of a cinema hall.
- GUI application (gui_app.py) – a Tkinter window that renders a live 10×7 seat grid, colour-coded green/red, with click-to-book interaction and automatic primary re-discovery on node failure.

Three stress-test scripts exercise the system under load:

- stress_test_1.py: Single-threaded: 100 sequential booking attempts on the same seat to confirm only one succeeds.
- stress_test_2.py: 5 concurrent threads each making random booking attempts with a small simulated delay.
- stress_test_3.py: 2 aggressive threads race to book all 70 seats simultaneously; verifies every seat ends up booked exactly once.

2. Database Schema

The system uses a single MongoDB database (cinema_db) with one collection (seats).

Collection: seats

Each document represents one physical seat:

{ "_id": 1, "is_booked": false, "customer_name": null, "version": 1 }

- _id: Integer (1–70) - Unique seat number; serves as the primary key.
- is_booked: Boolean - true when the seat has been reserved.
- customer_name: String / null - Name of the person who booked the seat; null if available.
- version: Integer - Optimistic-concurrency version counter (reserved for future use).

Replica Set Topology (rs0)

- Primary (elected): mongo1 (External port: 27017)
- Secondary: mongo2 (External port: 27018)
- Secondary: mongo3 (External port: 27019)

All writes go to the current primary. Secondaries replicate asynchronously and are promoted automatically if the primary becomes unavailable.

Atomic Booking Pattern

Double-booking is prevented without any application-level locking by using a conditional update:

collection.update_one({"_id": seat_id, "is_booked": False}, # filter: only match if still free {"$set": {"is_booked": True, "customer_name": name}})

If modified_count == 0 the seat was already taken between the read and the write, and the client is informed immediately.

3. Problems Encountered

3.1 Replica Set Initialisation Timing The mongo-init container needs all three database nodes to be fully running before starting the replica set. I initially just used a sleep 5 command to wait, but on my laptop, which is pretty slow, the setup still failed sometimes. When that happened, I just had to run a quick docker-compose restart mongo-init. I thought about writing a script to constantly check the status, but the manual restart was good enough for the testing environment.

3.2 Primary Discovery on Every Reconnect MongoDB usually handles finding the primary node automatically. However, since I mapped the nodes to different local ports (27017, 27018, 27019) instead of using proper DNS names, the automatic routing broke when a node failed. To fix this, I wrote a custom find_primary_port() function that manually pings all three ports to figure out which one is currently in charge.

3.3 GUI Connection Loss Handling When I stopped a Docker container to test how the system handles a crash, the GUI completely froze. This happened because the Python library was still trying to talk to the dead primary node. I fixed this by catching errors on every database call; if an error happens, our code automatically searches for the new primary node and tries the action again.